

Provable Σ_1 -Completeness, Machine-Checked: The Third Derivability Condition Derived inside Arithmetic

Charles C. Norton

September 2026

Abstract

We present a formalization, in the Rocq prover, of provable Σ_1 -completeness for a tower of arithmetic theories $T_0 \subseteq T_1 \subseteq \dots$ whose base T_0 has the axioms of Robinson arithmetic and the induction schema. For every level k and every Σ_1 formula σ with free variables $x_1, \dots, x_m$ we prove

$$T_0 \vdash \sigma(x_1, \dots, x_m) \rightarrow \text{Pr}_k(\ulcorner \sigma(\dot{x}_1, \dots, \dot{x}_m) \urcorner),$$

where Pr_k is the arithmetized provability predicate of T_k and the dotted substitution is computed inside arithmetic by an explicit formula for numeral substitution into codes. We also prove that this formula means what it says: at every valuation e it is true in $\mathbb{N}$ exactly when T_k proves σ with the numerals of $e(x_1), \dots, e(x_m)$ substituted. The third Hilbert–Bernays–Löb derivability condition $T_0 \vdash \text{Pr}_k(\ulcorner A \urcorner) \rightarrow \text{Pr}_k(\ulcorner \text{Pr}_k(\ulcorner A \urcorner) \urcorner)$ follows as the instance $\sigma := \text{Pr}_k(\ulcorner A \urcorner)$. The proof carries the textbook induction on Δ_0 formulas out inside the object theory, through a calculus of *provable instances* of code patterns whose slots hold numeral codes. The development adds about 12,800 lines to an existing formalization of the tower and its proof predicate, and the theorems depend on no axiom beyond excluded middle.

1 Introduction

Let T be an arithmetic theory with a Σ_1 provability predicate Pr_T . The Hilbert–Bernays–Löb derivability conditions [6, 9] are

- D1** if $T \vdash A$ then $T \vdash \text{Pr}_T(\ulcorner A \urcorner)$;
- D2** $T \vdash \text{Pr}_T(\ulcorner A \rightarrow B \urcorner) \rightarrow (\text{Pr}_T(\ulcorner A \urcorner) \rightarrow \text{Pr}_T(\ulcorner B \urcorner))$;
- D3** $T \vdash \text{Pr}_T(\ulcorner A \urcorner) \rightarrow \text{Pr}_T(\ulcorner \text{Pr}_T(\ulcorner A \urcorner) \urcorner)$.

Together with the diagonal lemma they yield Löb’s theorem, Gödel’s second incompleteness theorem, and the arithmetic soundness of the provability logic GL [2, 15]. D1 is a meta-level statement and follows from the Σ_1 -completeness of the theory for true sentences. D2 is an internal statement about concatenating two proofs. D3 is the deep one: it asks the theory to prove, about its own proof predicate, that whenever a proof exists this fact is itself provable. The standard route [2, 5] derives D3 from *provable* Σ_1 -completeness,

$$T \vdash \sigma(x_1, \dots, x_m) \rightarrow \text{Pr}_T(\ulcorner \sigma(\dot{x}_1, \dots, \dot{x}_m) \urcorner) \quad (\sigma \in \Sigma_1), \tag{1}$$

by taking $\sigma := \text{Pr}_T(\ulcorner A \urcorner)$, which is a Σ_1 sentence. Here $\dot{x}$ denotes the numeral of the value of x , so the right-hand side is the provability predicate applied to the code of σ with numerals substituted for its free variables, a term that the theory must compute.

The textbook proof of (1) is an induction on the structure of σ , and it is usually given in a few pages. Each step, however, asserts the existence of an object-level derivation of a statement *about* the proof predicate, and the derivation must be produced uniformly in the free variables. In a formalization nothing about the proof predicate is available except its definition: every syntactic computation the argument performs on codes (substitution, occurrence tests, the admissibility test for substitution, the construction of numerals) has to be carried out *inside* the object theory, on codes denoted by variables, and every inference that the informal argument makes “under Pr” has to be justified by an explicit object-level construction of a derivation. Abstract formal accounts of the incompleteness theorems accordingly take the derivability conditions as hypotheses [13].

This paper describes a complete machine-checked proof of (1) for every level of the arithmetic reflection tower of the **provability-verified** development [17], the arithmetic layer of the **tiling-verified** formalization of the polymodal provability calculus GLP* [18]. Our main results are the following, stated precisely in Section 3.

1. *Provable Σ_1 -completeness with free variables* (Theorem 3.3): for every level k and every Σ_1 formula σ , $T_0 \vdash \sigma \rightarrow \text{PrSub}_k(\sigma)$, where $\text{PrSub}_k(\sigma)$ applies Pr_k to the output of an explicit formalized numeral-substitution function.
2. *Adequacy* (Theorem 3.4): $\mathbb{N}, e \models \text{PrSub}_k(\sigma)$ if and only if T_k proves σ with the numerals of the values $e(x_i)$ substituted. The statement of the main theorem is therefore self-interpreting: a reader need not inspect the arithmetization to know what is proved.
3. *The sentence form and D3* (Corollaries 3.6 and 3.7): $T_0 \vdash \sigma \rightarrow \text{Pr}_k(\ulcorner \sigma \urcorner)$ for Σ_1 sentences, and $T_0 \vdash \text{Pr}_k(\ulcorner A \urcorner) \rightarrow \text{Pr}_k(\ulcorner \text{Pr}_k(\ulcorner A \urcorner) \urcorner)$.
4. *A proof architecture* that makes the internal induction tractable: code patterns with slots for numeral codes, a derived tag-3 substitution row for the code of any formula, and a calculus of provable instances with modus ponens, instantiation, existential elimination and case splitting (Section 4).

All results are proved in Rocq 9.0 and depend on no axiom other than the law of excluded middle. Together with the internal second derivability condition proved earlier in the same development, they make the derivability conditions theorems of T_0 rather than axioms.

Outline. Section 2 fixes the object theory, the coding and the proof predicate. Section 3 states the theorems. Section 4 explains the proof, from the rows of computation tables up to the substitution chain and the adequacy theorem. Section 5 describes the Rocq development and how to check it. Section 6 discusses related work, and Section 7 the limits of the result and the next steps.

2 The object theory

2.1 Language, substitution and formula classes

Terms and formulas are built from variables x_i ($i \in \mathbb{N}$), 0, successor S , $+$ and $\cdot$, with the primitive connectives $=$, $\perp$, $\rightarrow$, $\forall$ and $\exists$; negation, conjunction and disjunction are the usual abbreviations $\neg A \equiv A \rightarrow \perp$, $A \wedge B \equiv \neg(A \rightarrow \neg B)$ and $A \vee B \equiv \neg A \rightarrow B$. The numeral of n is $\bar{n} = S^n 0$.

Substitution $A[t/x]$ replaces the free occurrences of x in A by t without renaming bound variables. The calculus uses it only when it is admissible: $\text{adm}(x, t, A)$ holds when no free occurrence of x in A lies in the scope of a quantifier that binds a variable of t . Numerals are closed, so substituting a numeral is always admissible.

The relation $v < t$ is expressed as $\exists v' (v + Sv' = t)$ with $v' = x_{i+1}$ when $v = x_i$, and bounded quantifiers are

$$(\forall v < t) A \equiv \forall v (v < t \rightarrow A), \quad (\exists v < t) A \equiv \exists v (v < t \wedge A),$$

where v and its successor variable do not occur in t . The class Δ_0 is the least class containing the equations and $\perp$ and closed under $\rightarrow$ and bounded quantification; Σ_1 is the closure of Δ_0 under existential quantification.

2.2 The tower

The theories T_n are given by an inductive derivability relation **F0ProvesTn** n . Every level has the axioms of Robinson arithmetic **Q** and every instance of the induction schema

$$A[0/x] \rightarrow \forall x (A \rightarrow A[Sx/x]) \rightarrow \forall x A.$$

In addition, for every $k < n$ the level n has the local reflection schema $\text{Pr}_k(\ulcorner A \urcorner) \rightarrow A$, the schemes **D2** and **D3** for Pr_k , and the monotonicity scheme $\text{Pr}_k(\ulcorner A \urcorner) \rightarrow \text{Pr}_{k'}(\ulcorner A \urcorner)$ for $k \leq k' < n$. These level-restricted schemes are empty at $n = 0$, so the axioms of T_0 are exactly those of **Q** and induction. The rules are those of a Hilbert calculus for classical first-order logic with equality (the combinators **K** and **S**, double negation, modus ponens, generalization, equality and congruence axioms, universal instantiation and existential introduction at admissible substitutions, existential elimination, and distribution of $\forall$ over $\rightarrow$), together with a Löb rule for the level's own predicate: from $T_n \vdash \text{Pr}_n(\ulcorner A \urcorner) \rightarrow A$ infer $T_n \vdash A$. We return to this rule in Section 7.

The base development establishes the following facts, which we use.

Soundness each level is sound: its theorems are true in $\mathbb{N}$ under every valuation (**F0ProvesTn_sound**);

Cumulativity $T_n \vdash A$ implies $T_m \vdash A$ for $n \leq m$ (**F0ProvesTn_cumulative**);

Adequacy $\mathbb{N} \models \text{Pr}_k(\ulcorner A \urcorner)$ if and only if $T_k \vdash A$ (**F0ProvSentence_sat_iff**);

D1 $T_k \vdash A$ implies $T_m \vdash \text{Pr}_k(\ulcorner A \urcorner)$ for every m (**FOHBL3_provable**), obtained from the Σ_1 -completeness of **Q** for true closed Σ_1 sentences;

D2 $T_0 \vdash \text{Pr}_k(\ulcorner A \rightarrow B \urcorner) \rightarrow (\text{Pr}_k(\ulcorner A \urcorner) \rightarrow \text{Pr}_k(\ulcorner B \urcorner))$ (**FOHBL2_internal**), derived inside T_0 by merging two checked derivations and their computation tables.

2.3 Gödel numbering

Codes use the Cantor pairing $\langle a, b \rangle = \frac{1}{2}(a+b)(a+b+1) + b$, with terms on the left and formulas on the right:

$$\begin{aligned} \ulcorner x_i \urcorner &= \langle 0, i \rangle, & \ulcorner t = u \urcorner &= \langle 0, \langle \ulcorner t \urcorner, \ulcorner u \urcorner \rangle \rangle, \\ \ulcorner 0 \urcorner &= \langle 1, 0 \rangle, & \ulcorner \perp \urcorner &= \langle 1, 0 \rangle, \\ \ulcorner St \urcorner &= \langle 2, \ulcorner t \urcorner \rangle, & \ulcorner A \rightarrow B \urcorner &= \langle 2, \langle \ulcorner A \urcorner, \ulcorner B \urcorner \rangle \rangle, \\ \ulcorner t + u \urcorner &= \langle 3, \langle \ulcorner t \urcorner, \ulcorner u \urcorner \rangle \rangle, & \ulcorner \forall x_i A \urcorner &= \langle 3, \langle i, \ulcorner A \urcorner \rangle \rangle, \\ \ulcorner t \cdot u \urcorner &= \langle 4, \langle \ulcorner t \urcorner, \ulcorner u \urcorner \rangle \rangle, & \ulcorner \exists x_i A \urcorner &= \langle 4, \langle i, \ulcorner A \urcorner \rangle \rangle. \end{aligned}$$

The graph of pairing is defined by a Δ_0 formula.

2.4 Computation tables and the proof predicate

The syntactic operations on codes are represented by *computation tables*. A table is a list of rows (t, a_1, a_2, a_3, r) stored in five columns through Gödel's β -function $\beta(c, d, i) = c \bmod (d(i+1)+1)$, so that a table of length ℓ is given by eleven numbers. The tag t selects the operation the row records:

tag	the row (t, a_1, a_2, a_3, r) records
0	$r = 1$ if the variable x_{a_1} occurs in the term coded a_2 , else $r = 0$
1	$r = 1$ if x_{a_1} occurs free in the formula coded a_2 , else $r = 0$
2	$r = \lceil t[s/x_{a_1}] \rceil$ when $a_2 = \lceil s \rceil$ and $a_3 = \lceil t \rceil$
3	$r = \lceil A[s/x_{a_1}] \rceil$ when $a_2 = \lceil s \rceil$ and $a_3 = \lceil A \rceil$
4	$r = 1$ if $\text{adm}(x_{a_1}, s, A)$ when $a_2 = \lceil s \rceil$ and $a_3 = \lceil A \rceil$, else $r = 0$
5	$r = \lceil \overline{a_1} \rceil$

A table is *valid* when every row satisfies the step clause for its tag; the step clause decomposes the codes by pairing and refers to the rows of the sub-computations, which must occur in the same table. Validity is Δ_0 in the eleven numbers. Two facts connect tables to the operations they represent: in a valid table every row is correct (**tbl_sound**), and every finite set of rows closed under the step clauses is the set of rows of some valid table (**table_realize**). We write

$$\text{Row}_t(a_1, a_2, a_3, r) \equiv \exists T (\text{Valid}(T) \wedge (t, a_1, a_2, a_3, r) \in T),$$

a Σ_1 formula (**FOTBLEX**). Numeral codes are captured by

$$\text{Num}(z, \mu) \equiv \text{Row}_5(z, 0, 0, \mu) \wedge \forall u \forall s \text{Row}_2(u, s, \mu, \mu) \wedge \forall u \text{Row}_0(u, \mu, 0, 0),$$

which says that μ is the code of $\bar{z}$ and records, as rows, that substitution into μ changes nothing and that no variable occurs in it (**FONUMR**).

The proof predicate of level k is a Σ_1 formula $\text{Prf}_k(u, c)$ (**FOPRMAT**): there are a valid table, a β -coded sequence of formula codes $f_0, \dots, f_\ell$ and a sequence of justification codes such that $f_\ell = c$, each entry is justified as an axiom of T_k or as the conclusion of a rule applied to earlier entries, the syntactic side conditions of each justification are read from rows of the table, and each entry passes a guard. The parameter u is the code of the template itself; it is used to check the justifications of Löb-rule entries and, through a list of self-applied templates for the levels below k , of the level-restricted axioms. The level- k provability formula is

$$\text{Pr}_k(c) \equiv \text{Prf}_k(\lceil \overline{\text{Prf}_k} \rceil, c),$$

and $\text{Pr}_k(\lceil A \rceil)$ denotes its instance at the numeral $\lceil A \rceil$ (**FOProvSentence**).

3 Statements

3.1 Formalized numeral substitution

The right-hand side of (1) applies Pr_k to a code that depends on the values of the free variables. The language has no function symbol for substitution, so the code is described relationally. The description has to be capture-free for every Σ_1 formula σ , whatever the names of its variables; in particular the terms standing for the values must not be captured by the bound variables of Row and Num . We therefore let the values enter only through equations.

Definition 3.1 (Numeral substitution into a code). Let q be a variable index, c and r terms, $x_1, \dots, x_m$ variable indices and $y_1, \dots, y_m$ terms. Define $\text{Sub}_q(c; \vec{x}; \vec{y}; r)$ by recursion on m :

$$\text{Sub}_q(c; ; ; r) \equiv c = r,$$

$$\text{Sub}_q(c; x\vec{x}; y\vec{y}; r) \equiv \exists a \exists \mu \exists z (z = y \wedge \text{Num}(z, \mu) \wedge \text{Row}_3(\bar{x}, \mu, c, a) \wedge \text{Sub}_{q+3}(a; \vec{x}; \vec{y}; r)),$$

where a, μ and z are the variables x_q, x_{q+1} and x_{q+2} (**FOSUBNUMS**).

Each step substitutes, for the variable x , the numeral of the value of y into the formula coded by the current code c : z carries the value, μ is the code of its numeral, and the tag-3 row gives the code a of the result. The terms $\vec{y}$ occur only in the equations $z = y$, so no binder of Row or Num can capture them, and the chain variables $x_q, x_{q+1}, \dots$ are chosen above every variable of σ .

Definition 3.2 (Substituted provability). For a formula σ let $x_1 < \dots < x_m$ be its free variables, let M be the largest index of a variable occurring in σ , and let $Q = 2001 + M$. Define

$$\text{PrSub}_k(\sigma) := \exists x_Q (\text{Sub}_{Q+1}(\overline{\ulcorner \sigma \urcorner}; \vec{x}; x_1, \dots, x_m; x_Q) \wedge \text{Pr}_k(x_Q))$$

(F0SubProv). In the notation of (1), $\text{PrSub}_k(\sigma)$ is $\text{Pr}_k(\ulcorner \sigma(\dot{x}_1, \dots, \dot{x}_m) \urcorner)$.

The offset 2001 keeps the code variable and the chain away from the variable ranges reserved for the internal bound variables of the arithmetized operations, which all lie below 1000.

3.2 Main results

For a formula A , variables $x_1, \dots, x_m$ and terms $t_1, \dots, t_m$ write $A[t_1/x_1, \dots, t_m/x_m]$ for the sequential substitution $(\dots(A[t_1/x_1])\dots)[t_m/x_m]$ (F0substs); for closed t_i it coincides with simultaneous substitution.

Theorem 3.3 (Provable Σ_1 -completeness). *For every level k and every Σ_1 formula σ ,*

$$T_0 \vdash \sigma \rightarrow \text{PrSub}_k(\sigma).$$

Theorem 3.4 (Adequacy of the substituted predicate). *For every level k , every formula σ with free variables $x_1 < \dots < x_m$, and every valuation e ,*

$$\mathbb{N}, e \models \text{PrSub}_k(\sigma) \iff T_k \vdash \sigma[\overline{e(x_1)}/x_1, \dots, \overline{e(x_m)}/x_m].$$

Theorem 3.5 (Totality of the substitution). *For every formula σ ,*

$$T_0 \vdash \exists x_Q (\text{Sub}_{Q+1}(\overline{\ulcorner \sigma \urcorner}; \vec{x}; \vec{x}; x_Q) \wedge x_Q = x_Q).$$

Corollary 3.6 (Sentence form). *For every Σ_1 sentence σ , $T_0 \vdash \sigma \rightarrow \text{Pr}_k(\ulcorner \sigma \urcorner)$.*

Corollary 3.7 (Third derivability condition). *For every formula A and level k , $T_0 \vdash \text{Pr}_k(\ulcorner A \urcorner) \rightarrow \text{Pr}_k(\ulcorner \text{Pr}_k(\ulcorner A \urcorner) \urcorner)$.*

Corollary 3.7 is Corollary 3.6 at $\sigma := \text{Pr}_k(\ulcorner A \urcorner)$, which is a Σ_1 sentence. By cumulativity every statement proved in T_0 also holds in each T_n . The four results are collected in one Rocq theorem:

```
Theorem provable_sigma1_completeness : forall k,
  (forall A, F0sigma1 A -> F0ProvesTn 0 (F0ImplF A (F0SubProv k A))) /\
  (forall e A, F0sat e (F0SubProv k A) <->
    F0ProvesTn k (F0substs A (fvs A) (map (fun x => F0numeral (e x)) (fvs A)))) /\
  (forall A, F0sigma1 A -> (forall x, F0free_in x A = false) ->
    F0ProvesTn 0 (F0ImplF A (F0ProvSentence k A))) /\
  (forall A, F0ProvesTn 0
    (F0ImplF (F0ProvSentence k A) (F0ProvSentence k (F0ProvSentence k A)))).
```

Theorem 3.4 applies to every formula, not only to Σ_1 ones. Combined with soundness it shows that Theorem 3.3 is the familiar statement: at every valuation e satisfying σ , the theory T_k proves the numeral instance of σ at e . The formal content of Theorem 3.3 is stronger, since the implication is a single theorem of T_0 with free variables, not a family of statements about instances.

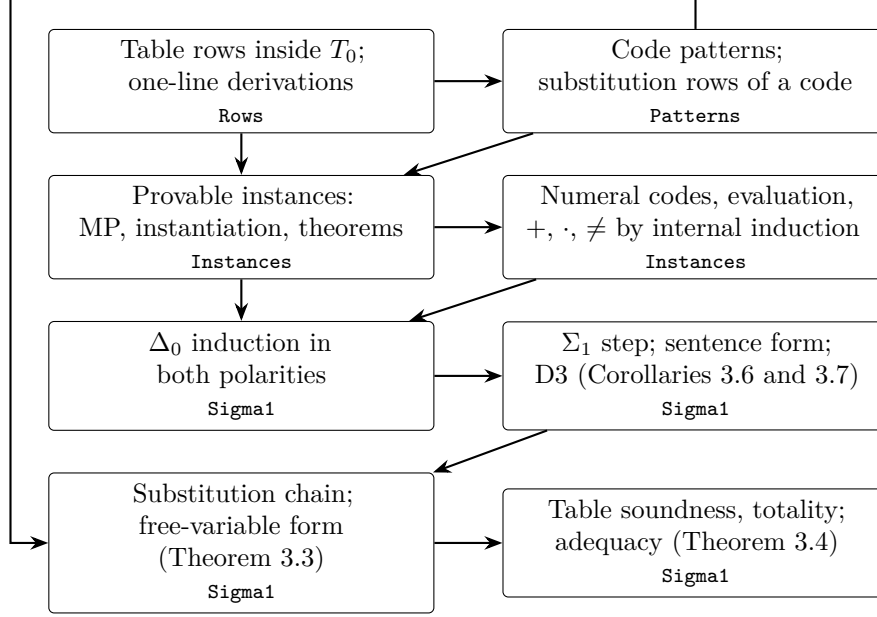


Figure 1: Structure of the proof and the Rocq file for each part.

4 The proof

Figure 1 shows the structure of the argument. We first derive, inside T_0 , the table rows that the argument needs (Section 4.1), then introduce code patterns and derive the substitution rows for the code of an arbitrary formula (Section 4.2). On top of these we build a calculus of provable instances (Section 4.3), in which the arithmetic of numerals is developed by internal induction (Section 4.4). The Δ_0 induction (Section 4.5) and the Σ_1 step (Section 4.6) give the sentence form and D3. The free-variable form follows from a chain of substitution rows (Section 4.7), and adequacy from the soundness of tables and the totality of the chain (Section 4.8).

Throughout, $G \vdash A$ denotes derivability in T_0 from a finite list G of hypotheses; the deduction theorem makes this equivalent to $T_0 \vdash \bigwedge G \rightarrow A$. Many of the lemmas below carry side conditions stating that certain variables are fresh; we omit them in the exposition and describe the discipline that discharges them in Section 5.

4.1 Rows derived inside the theory

The argument never evaluates a table: every row it uses is derived inside T_0 from rows of the sub-computations, by object-level constructions of tables. The basic operations are adding a row to a valid table whose step clause is met by rows already present, and joining the rows of two or three valid tables into one. Both rely on the object-level extension and concatenation of β -coded sequences proved in the base development.

The same machinery yields *one-line derivations*. If an axiom instance of T_k has a justification whose side conditions are rows (for instance, an instance of universal instantiation $\forall x \theta \rightarrow \theta[t/x]$ requires a tag-3 row for $\theta[t/x]$ and a tag-4 row with value 1), then T_0 derives Pr_k of the instance's code from those rows: a one-entry derivation, justified by the rows, witnesses the provability predicate. These derivations, D1 for closed theorems and the internal D2 are the only ways in which the argument introduces Pr_k .

4.2 Code patterns

Codes of formulas with numerals substituted for free variables are described by patterns with slots.

Definition 4.1 (Patterns). Patterns are generated by $p ::= \text{lit } n \mid \text{slot } i \mid \text{succ } p \mid \text{pair } pp$. Given a list of terms $\vec{e}$, the formula $\text{Pat}_B(\vec{e}, p, d)$ states that d is the value of p when $\text{slot } i$ is filled by e_i (FOPATF). The intermediate values of the pairs are named by bounded existential quantifiers over the variables $x_B, x_{B+1}, \dots$, so Pat is Δ_0 .

For a partial map ρ from variables to slot indices, the pattern $\pi_\rho(A)$ of a formula A is its code with each free variable x in the domain of ρ replaced by $\text{slot } \rho(x)$ (cpat_f). Every other variable contributes the literal code of the variable, and bound variables are removed from ρ under their binder. If the slots hold numeral codes $\ulcorner \bar{n}_i \urcorner$, the value of $\pi_\rho(A)$ is the code of A with the numerals substituted. Two elementary lemmas are used everywhere: every pattern has a code, $T_0 \vdash \exists d \text{Pat}_B(\vec{e}, p, d)$, and the code is unique (FOPrH_patf_total, FOPrH_patf_unique).

The central computational lemma derives the substitution row of a code from the patterns of the source and the target.

Lemma 4.2 (Substitution rows, FOPrH_rows_f). *Let $x \notin \text{dom}(\rho)$, let ρ' extend ρ by $x \mapsto |\vec{e}|$, and suppose G derives $\text{Num}(w_i, e_i)$ for every slot i of $\vec{e}$. Then*

$$G, \text{Pat}(\vec{e}, \pi_\rho(A), a), \text{Pat}(\vec{e}s, \pi_{\rho'}(A), c) \vdash \text{Row}_3(\bar{x}, s, a, c).$$

Proof sketch. By induction on A , and on terms for the equations, using the tag-2 rows for terms. At a slot the target and source codes coincide, and the Num hypothesis supplies the row stating that substitution into a numeral code leaves it unchanged. At the variable x itself the row returns s . At another literal variable y the step clause needs $\bar{y} \neq \bar{x}$, which is provable because $y \neq x$. At a binder the clause distinguishes whether the bound variable is x . The recursion is at the meta level and produces, for each A , an explicit object-level derivation. $\square$

4.3 Provable instances

The textbook induction proves statements of the form “if σ holds then $\text{Pr}(\ulcorner \sigma(\vec{x}) \urcorner)$ ”. In our setting the code $\ulcorner \sigma(\vec{x}) \urcorner$ is not a term, so we quantify over all codes matching the pattern.

Definition 4.3 (Provable instances, PRI). For a hypothesis list G , a pattern p and a list of numeral-code terms $\vec{e}$, $\text{PI}(G, p, \vec{e})$ holds if for every extension $G' \supseteq G$ and every term c ,

$$G' \vdash \text{Pat}(\vec{e}, p, c) \quad \text{implies} \quad G' \vdash \text{Pr}_k(c).$$

PI is a meta-level property of object-level derivability, and the quantification over extensions makes it stable under adding hypotheses, which the existential eliminations below require. The calculus has the following rules.

Lemma 4.4 (The calculus of provable instances). *Let the slots of $\vec{e}$ hold numeral codes under G .*

1. Theorems (*PRI_thm*). *If $T_0 \vdash A$ and the free variables of A lie in $\text{dom}(\rho)$, then $\text{PI}(G, \pi_\rho(A), \vec{e})$.*
2. Modus ponens (*PRI_mp*). *From $\text{PI}(G, \pi_\rho(A \rightarrow B), \vec{e})$ and $\text{PI}(G, \pi_\rho(A), \vec{e})$ infer $\text{PI}(G, \pi_\rho(B), \vec{e})$.*
3. Instantiation (*PRI_inst*). *If $\text{PI}(G, \pi_\rho(\forall x \theta), \vec{e})$ and $G \vdash \text{Num}(w, \mu)$, then $\text{PI}(G, \pi_{\rho'}(\theta), \vec{e}\mu)$ with $\rho' = \rho[x \mapsto |\vec{e}|]$.*
4. Existential elimination (*PRI_exe*). *If $G \vdash \exists x X$ and $\text{PI}(G \cup \{X[w/x]\}, p, \vec{e})$ for a fresh variable w , then $\text{PI}(G, p, \vec{e})$.*
5. Case split (*PRI_cases*). *If $\text{PI}(G \cup \{X\}, p, \vec{e})$ and $\text{PI}(G \cup \{\neg X\}, p, \vec{e})$, then $\text{PI}(G, p, \vec{e})$.*

6. Reslotting (*PRI_reslot*). If ρ and ρ' assign to each free variable of A slots whose contents are provably equal, then $\text{PI}(G, \pi_\rho(A), \vec{e})$ implies $\text{PI}(G, \pi_{\rho'}(A), \vec{e}')$.

Proof sketch. (1) By D1, $T_0 \vdash \text{Pr}_k(\ulcorner \forall \vec{x} A \urcorner)$; iterated instantiation (3) at the slot contents then gives the pattern of A . (2) The code of $A \rightarrow B$ is $\langle 2, \langle a, b \rangle \rangle$ for codes a and b of the patterns, which exist by totality; the internal D2 applied at these codes, with a guard row for the conclusion, gives $\text{Pr}_k(c)$. (3) The code of θ with the new slot is obtained from the code of $\forall x \theta$ by the tag-3 row of Lemma 4.2; the admissibility row has value 1 because the substituted code is a numeral code. A one-line derivation of the instantiation axiom and the internal D2 conclude. (4) and (5) follow from the corresponding rules of the calculus, because PI quantifies over extensions of G . (6) follows because patterns with provably equal slot contents have provably equal codes. $\square$

4.4 Arithmetic under the provability predicate

Numeral codes. T_0 proves $\forall z \exists \mu \text{Num}(z, \mu)$ by induction on z , the step adding the rows for $\ulcorner z + \bar{1} \urcorner = \langle 2, \ulcorner \bar{z} \urcorner \rangle$ to a table. Uniqueness follows by an internal induction, and inversion ($\text{Num}(0, \mu) \rightarrow \mu = \ulcorner 0 \urcorner$; $\text{Num}(Sz, \mu) \rightarrow \exists \mu' (\text{Num}(z, \mu') \wedge \mu = \langle 2, \mu' \rangle)$) by inverting the step clause of a tag-5 row.

Term evaluation. For every term t whose variables have slots, and every fresh value variable z whose slot holds the numeral code of t , $\text{PI}(G, \pi_\rho(t = z), \vec{e})$ (*PRI_teval*). The proof is by induction on t . Variables and 0 follow from theorems of the form $x = x$, successors from the congruence theorem, and sums and products from the next paragraph.

Sums, products and disequality by internal induction. The statement $\text{PI}(G, \pi(\bar{a} + \bar{b} = \bar{c}), \vec{e})$ for the numeral codes of values a, b and c with $a + b = c$ cannot be proved by a meta-level induction on b , because b is the value of a variable. We internalize PI for a fixed pattern as the formula

$$\text{PI}^b(\vec{e}, p) := \forall c (\text{Pat}(\vec{e}, p, c) \rightarrow \text{Pr}_k(c))$$

(*PRI*f) and prove, by the induction schema of T_0 , the formula

$$\Phi_+(N) := \forall \alpha \forall \beta (\text{Num}(N, \alpha) \rightarrow \text{Num}(x + N, \beta) \rightarrow \text{PI}^b(\mu_x \alpha \beta, \pi(x_0 + x_1 = x_2)))$$

for all N (*PhiOp*, *PRI_plus*). The base case uses the theorem $x + 0 = x$, and the step combines the induction hypothesis at the inverted numeral code with the theorem $x + y = z \rightarrow x + Sy = Sz$ under the provability predicate. Products are treated the same way over sums. Disequality of numerals, $\text{PI}(G, \pi(\neg \bar{a} = \bar{b}), \vec{e})$ for $a \neq b$, is proved by an internal induction on a with the theorems $0 \neq Sy$, $Sx \neq 0$ and $x \neq y \rightarrow Sx \neq Sy$ (*PhiNeq*, *PRI_neq*).

4.5 The Δ_0 induction

The induction over Δ_0 formulas has to handle formulas whose free variables are bound by quantifiers further out in σ . We keep, for every free variable x of the current subformula, a *holder* variable $h(x)$ of T_0 above all variables of σ , a slot $\rho(x)$, and a hypothesis $\text{Num}(h(x), e_{\rho(x)})$ in the context; $\text{hsub}_h(A)$ denotes A with each free x renamed to $h(x)$. The invariant *Inv* records these facts together with the freshness bounds.

Theorem 4.5 (Δ_0 induction, *D0P_all*). *For every Δ_0 formula A and every state $(G, h, \rho, \vec{e})$ satisfying the invariant,*

$$\begin{aligned} G \vdash \text{hsub}_h(A) &\implies \text{PI}(G, \pi_\rho(A), \vec{e}), \\ G \vdash \neg \text{hsub}_h(A) &\implies \text{PI}(G, \pi_\rho(\neg A), \vec{e}). \end{aligned}$$

Proof sketch. By induction on A , proving both polarities together.

Equations $t = u$. Choose fresh value variables for t and u with numeral codes by existential elimination. Term evaluation gives $\text{PI}(\pi(t = z_1))$ and $\text{PI}(\pi(u = z_2))$. In the positive case $G \vdash z_1 = z_2$, and the theorem $t = z \rightarrow u = z \rightarrow t = u$ concludes. In the negative case the values differ, disequality gives $\text{PI}(\pi(\neg z_1 = z_2))$, and the theorem $t = z_1 \rightarrow u = z_2 \rightarrow \neg z_1 = z_2 \rightarrow \neg t = u$ concludes.

$\perp$. The positive case is vacuous by ex falso; the negative case is the theorem $\neg \perp$.

Implication $B \rightarrow C$. In the positive case, split on $\text{hsub}(B)$ inside the theory (Lemma 4.4(5)). If it holds, $G \vdash \text{hsub}(C)$, and the theorem $C \rightarrow (B \rightarrow C)$ applies. If it fails, the negative hypothesis for B gives $\text{PI}(\pi(\neg B))$, and the theorem $\neg B \rightarrow (B \rightarrow C)$ applies. The negative case uses $B \rightarrow \neg C \rightarrow \neg(B \rightarrow C)$.

Bounded universals. If $G \vdash (\forall v < t) \text{hsub}(D)$, we prove by the induction schema of T_0 , on the numeral bound N , the internal statement

$$\Psi(N) := \forall \mu (\text{Num}(N, \mu) \rightarrow (\forall v < N) \text{hsub}(D) \rightarrow \text{PI}^\flat(\vec{e}\mu, \pi((\forall v < y) D))),$$

where y is a fresh variable with the new slot (**BALL_GEN**). The base case uses the theorem $(\forall v < 0) D$. The step applies the induction hypothesis for D at $v := N$, renamed to a slot by **cpat_f_subst_var**, and the theorem $D[y/v] \rightarrow (\forall v < y) D \rightarrow (\forall v < Sy) D$. Term evaluation for t and the Leibniz theorem $t = y \rightarrow Q(y) \rightarrow Q(t)$ then transfer the result from y to t . The negative case of bounded existentials is the same argument for $\neg D$.

Bounded existentials. If $G \vdash (\exists v < t) \text{hsub}(D)$, eliminate the witness, compute numeral codes for it and for the difference to t , derive $v + Sw = t$ under the predicate by term evaluation, apply the induction hypothesis for D , and conclude with a theorem introducing the bounded existential (**WITNESS**). The negative case of bounded universals is the same argument for $\neg D$. $\square$

4.6 The Σ_1 step, the sentence form and D3

Theorem 4.6 (Σ_1 induction, **S1P_all**). *For every Σ_1 formula σ and every state satisfying the invariant, $G \vdash \text{hsub}_h(\sigma)$ implies $\text{PI}(G, \pi_\rho(\sigma), \vec{e})$.*

Proof. For Δ_0 formulas this is the positive half of Theorem 4.5. For $\exists x A$, eliminate the witness into a fresh holder w_0 , choose a numeral code w_1 for it, and extend the state by the slot for x . The induction hypothesis gives PI for A over $\vec{e}w_1$. The theorem $A \rightarrow \exists x A$, instantiated at the slots, and modus ponens give PI for $\exists x A$ over $\vec{e}w_1$. Reslotting removes the slot of x , which is not free in $\exists x A$. $\square$

For a Σ_1 sentence σ take the context $[\sigma]$, the identity holder map and no slots. The pattern of a closed formula with no slots is its literal code, so $\text{PI}([\sigma], \pi(\sigma), [])$ at the code $\overline{\sigma}$ gives $[\sigma] \vdash \text{Pr}_k(\ulcorner \sigma \urcorner)$, and the deduction theorem gives Corollary 3.6. Corollary 3.7 is its instance at $\text{Pr}_k(\ulcorner A \urcorner)$, which is a closed Σ_1 formula.

4.7 The substitution chain and the free-variable form

For a formula σ with free variables $\vec{x}$ we work in the context $[\text{hsub}_h(\sigma)]$ with holders $h(x) = H + x$ above the chain. Existential elimination supplies numeral codes μ_i with $\text{Num}(h(x_i), \mu_i)$, and Theorem 4.6 gives $\text{PI}(\pi_\rho(\sigma), \vec{\mu})$ for the map ρ that sends x_i to slot i . By totality of patterns we name a code c of $\pi_\rho(\sigma)$ over $\vec{\mu}$, and PI yields $\text{Pr}_k(c)$. It remains to connect c to the chain of Definition 3.1.

Lemma 4.7 (The chain, SUBNUMS_chain). *Suppose G derives Num facts for the slots of $\vec{e}$ and $\text{Num}(y_i, \mu_i)$ for each i , together with $\text{Pat}(\vec{e}, \pi_\rho(A), a)$ and $\text{Pat}(\vec{e}\vec{\mu}, \pi_{\rho[\vec{x} \mapsto |\vec{e}|, \dots]}(A), c)$, where the variables $\vec{x}$ are outside $\text{dom}(\rho)$. Then $G \vdash \text{Sub}_q(a; \vec{x}; \vec{y}; c)$.*

Proof. By induction on the list $\vec{x}$. For the empty list, uniqueness of pattern codes gives $a = c$. For $x\vec{x}$, name the code a' of the pattern with the slot of x filled by existential elimination. Lemma 4.2 gives $\text{Row}_3(\vec{x}, \mu_1, a, a')$, and the induction hypothesis at a' gives the rest of the chain. The three existential quantifiers of the step are then introduced with the witnesses a' , μ_1 and y_1 . $\square$

Applied with ρ empty, $\vec{e}$ empty and the literal code $a = \lceil \sigma \rceil$, the lemma gives $\text{Sub}(\lceil \sigma \rceil; \vec{x}; h(\vec{x}); c)$ in the context $[\text{hsub}_h(\sigma)]$. Existential introduction at c then gives the holder form of $\text{PrSub}_k(\sigma)$, and the deduction theorem gives $T_0 \vdash \text{hsub}_h(\sigma) \rightarrow \text{PrSub}_k(\sigma)[h(\vec{x})/\vec{x}]$. Finally the holders are renamed back to the variables one at a time, by generalization and instantiation (`rename_all`). Each instantiation is admissible because the free occurrences of $h(x)$ sit exactly where x occurs free in σ , and in PrSub only inside the equations $z = y$ of the chain. This proves Theorem 3.3.

4.8 Adequacy

Proposition 4.8 (Soundness of the chain, SUBNUMS_sound). *If $\mathbb{N}, e \models \text{Sub}_q(c; \vec{x}; \vec{y}; r)$ and $e(c) = \lceil A \rceil$, then $e(r) = \lceil A[e(y_1)/x_1, \dots, e(y_m)/x_m] \rceil$.*

Proof. A true Row_t fact provides a valid table containing the row, and table soundness gives the row's meaning. Along the chain, the tag-5 row inside Num identifies μ as the code of the numeral of the value of y , and the tag-3 row identifies the next code as the code of the substituted formula. The chain variables are fresh for c, r and $\vec{y}$, so the updates of the valuation along the chain do not change their values. $\square$

Theorem 3.5 is proved like Theorem 3.3, with the trivially derivable hypothesis $\sigma \rightarrow \sigma$ in place of σ and the equation $x_Q = x_Q$ in place of $\text{Pr}_k(x_Q)$. To prove Theorem 3.4, note first that $\mathbb{N}, e \models \text{Pr}_k(x_Q)$ if and only if $\mathbb{N}, e \models \text{Pr}_k(\overline{e(x_Q)})$, by the semantic substitution lemma. If $\text{PrSub}_k(\sigma)$ is true at e , the witness for x_Q is the code of the numeral instance by soundness of the chain, so Pr_k of that code is true, and adequacy of Pr_k gives the T_k -derivation. Conversely, soundness of T_0 applied to Theorem 3.5 provides a witness for the chain at e , which is the code of the numeral instance, and adequacy of Pr_k turns the assumed T_k -derivation into the truth of Pr_k at that witness.

5 The formalization

5.1 Files and checking

The work consists of the last four files in the `theories` directory of the `provability-verified` development [17], placed after the files that define the syntax, the semantics, the proof checker and the internal second derivability condition (Table 1). With Rocq 9.0, `make check` builds the development and re-validates it with `rocqchk`, which is what the repository's continuous integration runs. Print `Assumptions provable_signal_completeness` reports the single axiom `Classical_Prop.classic`.

5.2 Engineering

Derivations under hypotheses. All object-level reasoning is done in the relation $\text{FOPrH } n \ G \ A$, derivability of A in T_n from a list G of hypotheses. It is defined as provability of

File	Lines	Stmts	Time	Content
Rows	2,952	98	987 s	tables holding given rows, guard rows, one-line derivations, step clauses from component facts
Patterns	2,892	84	133 s	patterns of terms and formulas, their totality and uniqueness, substitution, occurrence and capture rows of a code
Instances	3,769	146	240 s	provable instances and their calculus, numeral codes, term evaluation, sums, products, disequality
Sigma1	3,181	107	34 s	Δ_0 and Σ_1 inductions, sentence form, D3, substitution chain, adequacy, the summary theorem
total	12,794	435	1,394 s	

Table 1: The files of this work, with their **Theorem**, **Lemma** and **Corollary** declarations and their compilation times with `coqc` on a laptop.

the iterated implication and comes with introduction and elimination rules for the connectives and quantifiers, including existential elimination at a fresh name and the induction rule. This lets the Rocq proofs follow the informal natural-deduction style of Section 4.

Variable discipline. Naive substitution and fixed internal variable names make freshness the dominant source of side conditions. The development partitions the variables. The internal bound variables of the arithmetized operations lie below 1000: the tables and their lookups use the indices 2 to 50, the universal closures in Num the indices 802 to 804, and the provability matrix the indices below 252. Pattern bases, the variables of the internal inductions and the holders use indices of at least 1000, allocated upward from a bound V that grows as fresh names are taken. Most side conditions are then linear inequalities between indices, discharged by `lia`. The rest (a variable does not occur in a term, is not free in a formula, or a substitution is admissible) are discharged by a small family of tactics that decompose formulas built from the arithmetized operations and look up bounds stated as hypotheses.

Derivations as data. Object-level derivability is the inductive relation `F0ProvesTn`, so each theorem of the form $T_0 \vdash A$ denotes a derivation tree in the Hilbert calculus of Section 2. The meta-level inductions of Section 4 are ordinary Rocq inductions that build these trees, uniformly in the formula they are applied to.

6 Related work

Shankar [14] checked Gödel’s first incompleteness theorem with the Boyer–Moore prover, and O’Connor [10] verified the essential incompleteness of arithmetic in Coq. Neither needs the derivability conditions. Paulson [11, 12] formalized both incompleteness theorems in Isabelle/HOL for the theory of hereditarily finite sets, following Świerczkowski [16]. His proof of the second theorem establishes the derivability conditions for that theory, including an internal form of Σ -completeness. For arithmetic, the Lean 4 library Foundation [4] proves the second incompleteness theorem for Δ_1 -definable theories extending IS_1 , with formalized Σ_1 -completeness, including its form for formulas with free variables, and the third derivability

condition $\models_{\Sigma_1} \vdash \Box\sigma \rightarrow \Box\Box\sigma$. Its proofs reason inside an arbitrary model of $\models_{\Sigma_1}$, in which syntax and provability are internalized, and pass to derivability through the completeness theorem. The present development is proof-theoretic instead: every step is an explicit derivation in the Hilbert calculus of T_0 , carried out on codes denoted by object-level variables, with the provability predicate defined through β -coded computation tables. It also covers a tower of predicates Pr_k , states the numeral substitution as an explicit formula whose standard-model meaning is proved, and is written in Rocq. Popescu and Traytel [13] give an abstract account of the incompleteness theorems that takes the derivability conditions as hypotheses and instantiate it with Paulson’s development. Kirst and Hermes [7] and Kirst and Peters [8] develop synthetic approaches to undecidability and incompleteness in Coq, which avoid arithmetized proof predicates altogether.

On the mathematical side, the derivability conditions go back to Hilbert and Bernays [6] and Löb [9], arithmetization in a general setting to Feferman [3], and the proof of provable Σ_1 -completeness we follow is the standard one of the textbooks [2, 15, 5]. The tower $T_0 \subseteq T_1 \subseteq \dots$ is the arithmetic setting of the polymodal provability logics studied by Japaridze and Beklemishev [1], and the development it belongs to is motivated by the Löbian obstacle for self-modifying agents [19].

7 Limits and next steps

The Löb rule. The calculus of the tower, as defined in the base development, closes each level under a primitive Löb rule for its own predicate. The rule is sound, since every theorem of every level is true in $\mathbb{N}$, and the axioms of T_0 are exactly those of $\mathbf{Q}$ and induction. The statement of Theorem 3.3 is nevertheless about T_0 as defined, including the rule, and our development does not certify that the derivations it constructs avoid it. The next stage of the development formalizes the diagonal lemma, Löb’s theorem and the internal monotonicity $\text{Pr}_k(\ulcorner A \urcorner) \rightarrow \text{Pr}_{k'}(\ulcorner A \urcorner)$ for $k \leq k'$. It then removes the primitive rule and the level-restricted D2, D3 and monotonicity schemes, the first two of which are already redundant by the internal proofs, so that T_0 is Peano arithmetic and T_{n+1} is T_n plus local reflection for T_n . The present results are stated for the tower as it is defined now, and they will be re-established for the reduced tower.

Scope. The meaning theorem (Theorem 3.4) makes the formalized substitution trustworthy without inspecting the tables. The truth of the main theorem’s statement still rests on the definitions of the object theory, the coding and the Σ_1 class, which a reader has to check against their informal counterparts. These definitions are short and are collected in the file `theories/Syntax.v`.

8 Conclusion

Provable Σ_1 -completeness is where the arithmetization of syntax has to be used *inside* the theory rather than merely defined, and it is the step that informal expositions most often compress. We have given a complete machine-checked proof for every level of an arithmetic reflection tower, with free variables handled by an explicit numeral-substitution formula whose meaning is itself proved. The third derivability condition follows as a corollary. Code patterns with numeral-code slots, and the calculus of provable instances built on them, reduce the internal induction to local reasoning steps. With D2 proved earlier, D1 is a derived rule of every level, and D2 and D3 are theorems of the base theory.

References

- [1] L. D. Beklemishev. Reflection principles and provability algebras in formal arithmetic. *Russian Mathematical Surveys*, 60(2):197–268, 2005.
- [2] G. Boolos. *The Logic of Provability*. Cambridge University Press, 1993.
- [3] S. Feferman. Arithmetization of metamathematics in a general setting. *Fundamenta Mathematicae*, 49:35–92, 1960.
- [4] FormalizedFormalLogic. Foundation: formalization of mathematical logic in Lean 4. <https://github.com/FormalizedFormalLogic/Foundation>, 2023–2026.
- [5] P. Hájek and P. Pudlák. *Metamathematics of First-Order Arithmetic*. Springer, 1993.
- [6] D. Hilbert and P. Bernays. *Grundlagen der Mathematik*, volume II. Springer, 1939.
- [7] D. Kirst and M. Hermes. Synthetic undecidability and incompleteness of first-order axiom systems in Coq. In *12th International Conference on Interactive Theorem Proving (ITP 2021)*, volume 193 of *LIPIcs*, 2021.
- [8] D. Kirst and B. Peters. Gödel’s theorem without tears: essential incompleteness in synthetic computability. In *31st EACSL Annual Conference on Computer Science Logic (CSL 2023)*, volume 252 of *LIPIcs*, 2023.
- [9] M. H. Löb. Solution of a problem of Leon Henkin. *The Journal of Symbolic Logic*, 20(2):115–118, 1955.
- [10] R. O’Connor. Essential incompleteness of arithmetic verified by Coq. In *Theorem Proving in Higher Order Logics (TPHOLs 2005)*, volume 3603 of *Lecture Notes in Computer Science*, pages 245–260. Springer, 2005.
- [11] L. C. Paulson. A machine-assisted proof of Gödel’s incompleteness theorems for the theory of hereditarily finite sets. *The Review of Symbolic Logic*, 7(3):484–498, 2014.
- [12] L. C. Paulson. A mechanised proof of Gödel’s incompleteness theorems using Nominal Isabelle. *Journal of Automated Reasoning*, 55(1):1–37, 2015.
- [13] A. Popescu and D. Traytel. A formally verified abstract account of Gödel’s incompleteness theorems. In *Automated Deduction – CADE 27*, volume 11716 of *Lecture Notes in Computer Science*, pages 442–461. Springer, 2019.
- [14] N. Shankar. *Metamathematics, Machines, and Gödel’s Proof*. Cambridge University Press, 1994.
- [15] C. Smoryński. *Self-Reference and Modal Logic*. Springer, 1985.
- [16] S. Świerczkowski. Finite sets and Gödel’s incompleteness theorems. *Dissertationes Mathematicae*, 422:1–58, 2003.
- [17] C. C. Norton. provability-verified: provability in arithmetic, machine-checked in Rocq. <https://github.com/CharlesCNorton/provability-verified>, 2026.
- [18] C. C. Norton. tiling-verified: a Rocq formalization of the polymodal provability calculus GLP* and its arithmetic tower. <https://github.com/CharlesCNorton/tiling-verified>, 2026.
- [19] E. Yudkowsky and M. Herreshoff. Tiling agents for self-modifying AI, and the Löbian obstacle. Technical report, Machine Intelligence Research Institute, 2013.

A Rocq statements

The definitions of the formalized substitution and of the substituted provability formula:

```

Fixpoint FOSUBNUMS (q : nat) (c : FOTerm) (xs : list nat) (ys : list FOTerm)
  (r : FOTerm) : FOFormula :=
  match xs, ys with
  | x :: xs', y :: ys' =>
    FOExists q (FOExists (S q) (FOExists (S (S q))
      (FOAnd (FOEq (FOVar (S (S q))) y)
        (FOAnd (FONUMR (FOVar (S (S q))) (FOVar (S q)))
          (FOAnd (FOTBLEX (FOnumeral 3) (FOnumeral x) (FOVar (S q)) c (FOVar q))
            (FOSUBNUMS (S (S (S q))) (FOVar q) xs' ys' r))))))
  | _, _ => FOEQ c r
  end.

Definition FOSubProv (k : nat) (A : FOFormula) : FOFormula :=
  SPF (subq A) (FOnumeral (FOcode_f A)) (fvs A) (map FOVar (fvs A))
  (FOPRu (FOPrCores k) (FOu0 k) (FOVar (subq A))).

```

Here $\text{SPF } Q \ c \ xs \ ys \ R$ is $\text{FOExists } Q \ (\text{FOAnd } (\text{FOSUBNUMS } (S \ Q) \ c \ xs \ ys \ (\text{FOVar } Q)) \ R)$, $\text{subq } A$ is $2001 + \text{FOvars_max } A$, $\text{fvs } A$ lists the free variables of A in increasing order, and $\text{FOPRu } (\text{FOPrCores } k) \ (\text{FOu0 } k) \ c$ is $\text{Pr}_k(c)$, which coincides with $\text{FOProvSentence } k \ A$ at $c = \text{FOnumeral } (\text{FOcode_f } A)$.

The provable instances of a pattern:

```

Definition PRI (n : nat) (cores : list nat) (u0 V : nat) (G : list FOFormula)
  (p : CPat) (env : list FOTerm) : Prop :=
  forall G' B c, (forall X, In X G -> In X G') -> FOctx_avoid G' 0 1000 ->
  FOTms_avoid [c] 0 1000 -> V <= B -> Clean G' [c] B ->
  FOPrH n G' (FOPATF B env p c) -> FOPrH n G' (FOPRu cores u0 c).

```

The Δ_0 and Σ_1 inductions:

```

Theorem DOP_all : forall n k W A, F0delta0 A -> DOP n k W A.
Theorem S1P_all : forall n k W A, F0sigma1 A -> S1P n k W A.

```